

EMERGING TECHNOLOGIES & APPLICATIONS

Internet of Compute: Fundamentals, Architecture, Key Technologies, and Scenario Experiments

Li Wei^{1,2,*}, Zhao Weibo², Wang Runyan², Sang Liu², Huang Jinfei², Sun Songlin¹, Liu Ruming², Su Yue², Ma Fei²

¹ Beijing University of Posts and Telecommunications, Beijing 100 876, China

² China Academy of Information and Communications Technology, Beijing 100 191, China

* The corresponding author, email: liwei3@caict.ac.cn

Cite as: W. Li, W. Zhao *et al.*, “Internet of Compute: Fundamentals, architecture, key technologies, and scenario experiments,” *China Communications*, 2026, vol. 23, no. 9, pp. 341–366, doi: 10.23919/JCC.fa.2026-0311.202609.

Abstract: Traditional Internet architectures struggle to perceive and schedule computing resources, while existing computing-network solutions suffer from closed control planes and inefficient transmission protocols. To address these limitations, we propose the Internet of Compute (IoC), a next-generation network architecture supporting cross-domain intelligent task scheduling. We introduce “resource state entropy” and a network capacity model as the theoretical foundation to quantify the dynamics and equilibrium of computing, network, and storage resources. Based on entropy balance, we construct a novel three-tier architecture featuring an independent Interconnection Resource Layer. Furthermore, we propose the Computing Resource Identifier (CRID) for unified resource positioning, the Resource-Defined Memory Access Protocol (RDMAP) for long-distance lossless transmission, and an entropy-balance scheduling algorithm for global resource matching. Experimental results in large model training and inference scenarios demonstrate that our approach significantly improves resource matching accuracy and cross-domain transmission efficiency, validating the feasibility of the IoC architecture.

Keywords: computing resource identifier (CRID); entropy balance; Internet of Compute (IoC); RDMAP; resource state entropy

I. INTRODUCTION

With the rapid development of technologies such as artificial intelligence (AI), big data, and edge computing, traditional Internet architectures are increasingly revealing limitations in supporting large-scale, low-latency, and high-reliability compute scheduling. Although the classic TCP/IP system successfully achieved global general-purpose data transmission, its design paradigm of “decoupling the bearer network from the service network [1], with no separation of the control plane and data plane [2]” makes it difficult to effectively perceive [3] and schedule [4] upper-layer computing resources [5]. In recent years, computing demand has evolved from centralized cloud data centers [6] to a ubiquitous cloud-edge-terminal distribution [7], necessitating a new generation of network architecture [8] capable of deep integration across computing, storage, and networking [9].

In this context, the Internet of Compute (IoC) emerged. This concept was first proposed by the China Academy of Information and Communications Technology (CAICT) in 2025 [10], aiming to build a next-generation architecture that supports intelligent scheduling of computing tasks across domains, subjects, and levels. Its core goal is to break the barriers between existing networks and computing through unified identification [11], programmable protocols [12], and collaborative scheduling mechanisms, achieving internet-level interconnection and efficient utilization

of computing resources. Based on previous work, this paper systematically expands the theoretical foundation of the IoC, proposes a unified architecture integrating the ternary relationship of computing, storage, and networking, and makes key technical breakthroughs in protocol design, resource identification, and scheduling algorithms.

The specific research content of this paper is summarized below.

- **Theoretical Expansion:** Improving the concept of the IoC to fully cover the relationship between computing, storage, and networking, and adding theories such as “network resource state entropy” and “network capacity models”.
- **Architectural Updates:** Constructing a new Interconnection Resource Layer based on the principle of resource state entropy balance, including transport control protocols and compute identifiers.
- **Technical Innovation:** Proposing the RDMAP protocol for long-distance lossless transmission and novel scheduling algorithms based on compute resource identification.
- **Scenario Use Cases:** Defining architectures and key technologies for two major scenarios: large model training and inference.
- **Experimental Verification:** Conducting experiments on compute identification, RDMAP, and scheduling algorithms in AI training and inference scenarios.

II. RELATED WORK

2.1 Evolution of Network Architecture

The Internet originated from the idea of packet switching in the 1960s, and the establishment of the TCP/IP protocol suite in the 1970s laid the foundation for its core feature of open interconnection. This system adopts a five-layer protocol stack, with the service network decoupled from the bearer network. The bearer network only provides general transmission capabilities, is unaware of upper-layer applications, and has not yet separated the control plane from the forwarding plane [13].

Subsequently, distributed computing emerged, mainly represented by grid computing and cloud computing. Grid computing focuses on cross-organizational collaboration of compute [14, 15], while cloud com-

puting emphasizes the pooling [16] and on-demand delivery [17] of IT resources [18]. Although cloud-edge collaboration and cloud services deeply integrate network capabilities at the commercial level, this integration remains superficial. The deep, native integration of computing and networking has not been realized at the protocol stack and scheduling mechanism levels, leading to persistent semantic gaps between task demands and resource provisioning.

In recent years, paradigms such as Compute Network, Converged Computing and Networking, Computing-First Network, and Computing-Aware Network (CAN) have attempted to integrate heterogeneous compute from the network side [19]. However, these existing solutions are fundamentally transport-centric extensions rather than architectural reinventions. They treat computing as terminal nodes attached to the network, relying on heuristic routing and tightly coupled, closed control planes. This closed nature makes it difficult to support flexible cross-subject and cross-regional compute scheduling. Meanwhile, the lack of low-latency [20] programmable transmission protocols [21] prevents the Internet-scale wide interconnection of compute resources [22] as an open resource pool.

NVIDIA put forward the vision of a global network of GPU compute. Through high-speed interconnection, a unified software stack, and cloud-edge-end collaboration, it integrates GPU resources distributed in global data centers, edge nodes, and end devices into a logically unified supercomputing platform. While this reflects the evolutionary trend of ubiquitous compute, it remains a vendor-specific, hardware-centric solution rather than an open, fundamental network architecture evolution.

2.2 Related Research on the Internet of Compute

To bridge the aforementioned gaps, Li Wei et al. from the China Academy of Information and Communications Technology (CAICT) first proposed the concept of the Internet of Compute (IoC) in 2025 [10]. Unlike CAN or Resource-Aware Networks, which primarily focus on steering traffic to available compute, IoC introduces a fundamental conceptual breakthrough: treating the “computing task” as the primary scheduling citizen, akin to data packets in the traditional Internet.

IoC aims to build a next-generation network archi-

ture that supports cross-domain scheduling of computing tasks. Its core innovation lies in introducing a unified compute identifier and protocol interface to enhance the interconnection of heterogeneous compute [23] and high-performance cross-domain transmission. By decoupling the control plane from the bearer network, IoC forms an independent logical interconnection resource layer. This architectural shift transforms the scheduling paradigm from network-driven traffic routing to task-driven resource matching, breaking through the limitations of existing paradigms in cross-subject scheduling and wide-area collaboration.

III. FUNDAMENTAL THEORIES OF THE INTERNET OF COMPUTE

Building upon previous architectural research, this paper systematically proposes a fundamental theoretical system for the IoC. To address the lack of algorithmic breakthroughs in existing distributed computing and resource abstraction, this theoretical system is anchored by the concept of Entropy Balance. It provides a mathematically quantifiable framework that fully covers task demand modeling, dynamic compute-network-storage resource capacity evaluation, and cross-domain collaborative scheduling mechanisms.

3.1 Definition of the Internet of Compute

The Internet of Compute is an enhanced evolutionary form of the Internet oriented toward computing tasks. It fundamentally shifts the operational logic from “host-to-host data transmission” to “task-to-resource matching.” By constructing a unified computing identifier and protocol interface, IoC strengthens heterogeneous compute and cross-domain high-performance transmission capabilities. This enables computing tasks and their associated data to be accurately matched with adaptive resources, thus forming a logically interconnected network with intelligent perception, real-time discovery, and on-demand acquisition capabilities.

The interconnected objects cover IT software and hardware resources such as computing, storage, and networking. The service objects are task-based applications, which are characterized by clear start–end boundaries, bursty and diverse resource demands—such as artificial intelligence inference, model training, scientific computing, and video rendering.

3.2 Demand Characteristics of Task-Based Applications

Different from persistent online services, task-based applications feature explicit input and output, resource release upon completion, strong demand burstness, obvious periodicity, and high diversity, relying heavily on computing–network collaboration.

Based on resource demand attributes, application forms, and deployment constraints, the fundamental theory of IoC must satisfy four core characteristics to distinguish itself from traditional heuristic-based resource allocation:

- **Elasticity**
Networks and compute shall dynamically adapt to task characteristics. In response to the discreteness of AI tasks, the system should schedule tasks and data flows on demand, and seamlessly expand to remote resources when local compute is insufficient. This breaks the siloed mode of rigid transmission in traditional networks and the isolated deployment of compute in cloud-edge paradigms, fully exploiting the lossless transmission capability of RDMA.
- **Predictable Capacity**
Traditional information entropy only optimizes information coding and cannot characterize the randomness of task–resource matching in dynamic networks. As an algorithmic breakthrough, this paper extends entropy theory to the compute dimension and proposes models of compute state entropy and network transmission entropy. The former quantifies task arrival rate and resource demand fluctuation, while the latter describes bandwidth utilization and delay jitter.
The mathematical coordination of the two enables accurate, proactive prediction of compute pools and network bandwidth. For instance, when batch training requires 100 GPUs, the system identifies the local shortage in advance via entropy evaluation, triggers cross-domain invocation, and synchronously reserves a 10 Gbps link. This fundamentally solves the problems of blind compute supply and idle network resources prevalent in existing resource-aware networks.
- **Resource Pooling**
Faced with the fragmentation of heterogeneous resources (CPU, GPU, TPU, etc.) caused by geo-

graphical and architectural differences, resource pooling abstracts distributed resources into feature vectors of computing, storage, and networking through unified identification. Unlike traditional distributed computing abstractions, IoC characterizes their dynamic states with the aforementioned entropy values, achieving unified management and reuse under a global logical view.

- Entropy Balance

In response to task diversity and resource heterogeneity, entropy balance serves as the core algorithmic mechanism that distinguishes IoC from traditional load balancing. It does not pursue absolute uniformity of node utilization, but seeks the optimal mathematical trade-off among system performance, resource efficiency, and quality of service.

For example, delay-sensitive inference tasks are preferentially assigned to low-latency, high-computing-performance nodes even if their loads are relatively high; throughput-prioritized training tasks are scheduled to lightly loaded clusters to maximize overall throughput. Its core principle is to match resources to tasks on demand based on quantified entropy states, thereby avoiding local congestion or idleness—a limitation that heuristic-based cloud-edge collaboration fails to overcome.

3.3 Computing Task Model

This paper denotes computing tasks into two independent random variables: The number of task arrivals per unit time follows a Poisson distribution with parameter λ ; The service time of a single task follows a normal distribution with parameter μ . Three typical task types are accordingly classified: AI training: low λ , high μ , belonging to long-duration and low-frequency tasks; AI inference: high λ , low μ , belonging to short-duration and high-frequency tasks; Video rendering: intermediate characteristics with balanced performance.

This model provides a statistical foundation for resource state modeling and scheduling strategy design. Its characteristic probability distributions are shown in Figure 1.

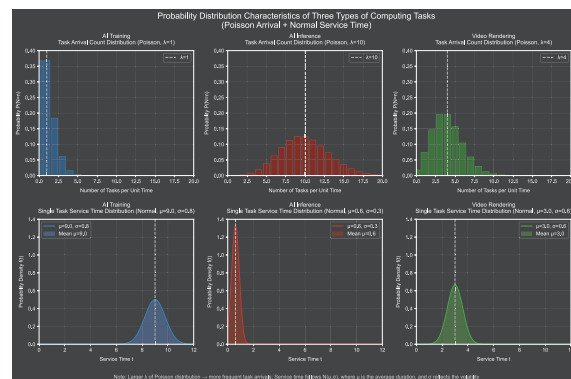


Figure 1. Probability distribution characteristics of three types of computing tasks.

3.4 Resource State Entropy: Quantification of Dynamics and Equilibrium

On the basis of the previous Computing Resource State Entropy, this paper further introduces Network State Entropy and Storage State Entropy, so as to construct a full-dimensional Resource State Entropy system.

Assume that the whole network contains multiple resource nodes, and the state of each node is composed of a feature vector including computing utilization rate, communication utilization rate, and storage utilization rate. Resource State Entropy is defined as a measure of the uncertainty of a node’s state: the more stable the state, the lower the entropy; the greater the fluctuation, the higher the entropy.

Let the network-wide resource matrix be R , which contains N resource nodes, with each resource pool denoted as r_i . The computing resource feature vector is defined as $\mathbf{r}_i(c_i, l_i, s_i)$, where c_i is the available computing state, l_i is the available communication state, and s_i is the available storage state, for $i = 1, 2, \dots, M$. This paper proposes the concept of “resource state entropy”: for the state of a node r_i , the more stable the state, the lower the entropy; conversely, the more unstable the state, the higher the entropy.

Now define the computing utilization of resource pool i at each hour in a day as c_i , where $k = 1, 2, \dots, 24$. The utilization here refers to workload utilization.

Let $P(c_{ik})$ be the probability that the average utilization appears at each hour of the day.

According to the definition of the entropy function [24], we have:

The “computing resource state entropy” of node c_i

is

$$H(c_i) = - \sum_{k=1}^{24} P(c_{ik}) \log (P(c_{ik})) . \quad (1)$$

Similarly, let the link bandwidth utilization rate of the node at the k -th moment be l_{ik} with probability $P(l_{ik})$, and the storage occupancy rate at the k -th moment be s_{ik} with probability $P(s_{ik})$. We can obtain:

The “network resource state entropy” of the node is

$$H(l_i) = - \sum_{k=1}^{24} P(l_{ik}) \log (P(l_{ik})) . \quad (2)$$

The “storage resource state entropy” of the node is

$$H(s_i) = - \sum_{k=1}^{24} P(s_{ik}) \log (P(s_{ik})) . \quad (3)$$

Therefore, the resource state of a node can be represented as an entropy vector

$$H_i = (H(c_i), H(l_i), H(s_i)) . \quad (4)$$

3.4.1 State Types and State Entropy

To characterize dynamic resource usage in the Internet of Compute, this paper models node states along two complementary dimensions: node type and utilization interval. Node type reflects the service and load characteristics of hosted tasks, while utilization intervals discretize continuous resource measurements for entropy calculation.

Directly using instantaneous utilization values would expand the state space and reduce model generalizability. Therefore, resource utilization is divided into four representative intervals: [0, 30%], [30%, 60%], [60%, 80%], and [80%, 100%], corresponding to idle, low-load, medium-load, and high-load states, respectively. Table 1 summarizes the resulting computing-resource entropy distribution.

Based on their task characteristics and resource usage patterns, four representative node types are defined as follows.

Type A represents inference-oriented nodes with short task cycles, light loads, transient resource usage, and elastic compute release. Their network traffic consists of short periods of high bandwidth and frequent establishment and termination of short connections. Storage access is dominated by random reads of

Table 1. State entropy of computing resources.

State Type	Utilization Rate	Probability	State Entropy
A (Inference-intensive, on-demand resource release)	[80%, 100%]	70%	1.1568
	[60%, 80%]	20%	
	[30%, 60%]	10%	
	[0, 30%]	0%	
B (Training-intensive, long duration, low arrival rate)	[80%, 100%]	90%	0.469
	[60%, 80%]	0%	
	[30%, 60%]	10%	
	[0, 30%]	0%	
C (Rendering, cloud desktops and other relevant workloads)	[80%, 100%]	50%	1
	[60%, 80%]	0%	
	[30%, 60%]	50%	
	[0, 30%]	0%	
D (Idle)	[80%, 100%]	0%	0
	[60%, 80%]	0%	
	[30%, 60%]	0%	
	[0, 30%]	100%	

small data blocks, particularly model weights, with few writes and concentrated hot spots that are well suited to caching.

Type B represents large-model training nodes with large task scales, long execution durations, and stable compute occupancy. Such nodes require persistent, high-bandwidth, and low-jitter connections for frequent gradient synchronization and parameter exchange, typically supported by high-performance protocols such as RDMA.

Type C represents real-time interactive workloads, including rendering, gaming, and cloud PCs. These workloads are GPU-intensive, highly concurrent, and latency-sensitive, requiring bidirectional, low-latency streaming transmission. Their storage follows a hybrid read–write pattern and requires sufficient capacity for resource preloading and real-time data writes.

Type D represents idle or low-utilization nodes that serve as elastic resource reserves for bursty task arrivals and global scheduling balance.

These node types are representative abstractions rather than an exhaustive classification of business workloads. In practical deployments, a node may host mixed workloads and transition among different states over time; such variations are precisely the dynamic characteristics that resource state entropy is intended

to quantify.

Table 2. State entropy of network resources.

State Type	Bandwidth Utilization Rate	Probability	State Entropy
A (Inference-intensive, on-demand resource release)	[80%, 100%]	10%	1.325
	[60%, 80%]	10%	
	[30%, 60%]	80%	
	[0, 30%]	0%	
B (Training-intensive, long duration, low arrival rate)	[80%, 100%]	90%	0.469
	[60%, 80%]	0%	
	[30%, 60%]	10%	
	[0, 30%]	0%	
C (Rendering, cloud desktops and other relevant workloads)	[80%, 100%]	30%	0.865
	[60%, 80%]	40%	
	[30%, 60%]	30%	
	[0, 30%]	0%	
D (Idle)	[80%, 100%]	0%	0
	[60%, 80%]	0%	
	[30%, 60%]	0%	
	[0, 30%]	100%	

Table 3. State entropy of storage resources.

State Type	Storage Utilization Rate	Probability	State Entropy
A (Inference-intensive, on-demand resource release)	[80%, 100%]	0%	1
	[60%, 80%]	0%	
	[30%, 60%]	50%	
	[0, 30%]	50%	
B (Training-intensive, long duration, low arrival rate)	[80%, 100%]	0%	0
	[60%, 80%]	100%	
	[30%, 60%]	0%	
	[0, 30%]	0%	
C (Rendering, cloud desktops and other relevant workloads)	[80%, 100%]	60%	0.96
	[60%, 80%]	20%	
	[30%, 60%]	20%	
	[0, 30%]	0%	
D (Idle)	[80%, 100%]	0%	0
	[60%, 80%]	0%	
	[30%, 60%]	0%	
	[0, 30%]	100%	

The corresponding state-entropy distributions of network and storage resources are summarized in Tables 2 and 3, respectively.

Combining the four representative node types with the four utilization intervals yields fine-grained resource-state combinations that support subsequent entropy calculation and scheduling decisions. Additional node types may be introduced according to deployment environments, business characteristics, and task requirements. Future work will further refine and validate the classification using measured resource data and practical scheduling results.

3.4.2 Global Entropy of States and Balanced Entropy

It can be seen from Table 1 that node resources are generally in a mixed state of Type A, Type B, Type C, and Type D, thus being in a state of entropy instability. For a complete resource node r_i , the comprehensive state entropy can be constructed as follows:

$$H(r_i) = w_c H(c_i) + w_l H(l_i) + w_s H(s_i), \quad (5)$$

where w_c , w_l , and w_s represent the weights of computing, network, and storage in resources, respectively, reflecting the degree of influence of each type of resource on service performance. They satisfy the following constraint: $w_c + w_l + w_s = 1$.

Weights can be flexibly adjusted by users according to service requirements. Taking the Type A state as an example, although inference tasks rely on computing resources, they feature non-sustained high load occupation and large fluctuations in computing entropy, so the computing weight should not be set too high. In contrast, network latency directly determines service response speed, and network stability is crucial, especially in edge scenarios. Therefore, inference tasks should be assigned relatively high and close weights for computing and network, while a low weight for storage, which only needs to meet occasional requirements such as model loading.

It should be noted that a lower node entropy value is not necessarily better. For instance, the Type D state has an entropy value of zero and high stability, but suffers from low resource utilization due to idleness. The ideal goal is to enable each node to maintain state stability while achieving high resource utilization, that is, to pursue an entropy level of high efficiency and balance. In an environment with strong task volatility

and high uncertainty, it is more important to comprehensively consider the distribution characteristics of the four states (Type A, Type B, Type C, and Type D).

To this end, a reference value can be set as the balanced entropy. In view of the fact that the Type B state corresponds to a high and stable resource utilization rate, its entropy value can be selected as the benchmark of balanced entropy. In the process of task scheduling, tasks are preferentially allocated to nodes whose state entropy is close to this balanced value. Through the dynamic calculation and optimization of the global network entropy, each node is guided to approach the balanced entropy after task allocation, thereby realizing the rational distribution of resources and effectively avoiding the excessive concentration of compute or large-scale resource idleness.

3.5 Entropy Balance for Resource Matching and Task Scheduling

Suppose computing service provider A has an inference task matrix T containing M tasks, with t_i denoting the i -th task. The objective is to match these tasks with network-wide computing resources and determine an optimal resource vector R that satisfies the requirements of t_i .

Owing to the uncertainty and volatility of $t_i(c_i, l_i, s_i)$, resource nodes may undertake tasks randomly in the absence of an effective matching mechanism. Figure 2 illustrates their initial distribution in the three-dimensional resource space, whose axes represent computing state c_j , communication state l_j , and storage state s_j , respectively. The dispersed and nonuniform distribution indicates potential mismatches between task requirements and resource states, resulting in disordered resource entropy.

In the IoC, entropy-balanced matching aims to minimize the impact of task uncertainty and volatility on the stable distribution of resource nodes in the resource space R . In particular, variations in task arrival frequency and service time should have minimal effects on resource-state stability. Achieving this objective requires network-wide task–resource matching and scheduling.

When task t_i is well aligned with resource node r_j in the resource space, the node remains relatively stable, resulting in low single-node entropy and a globally balanced state. Conversely, a substantial mismatch in-

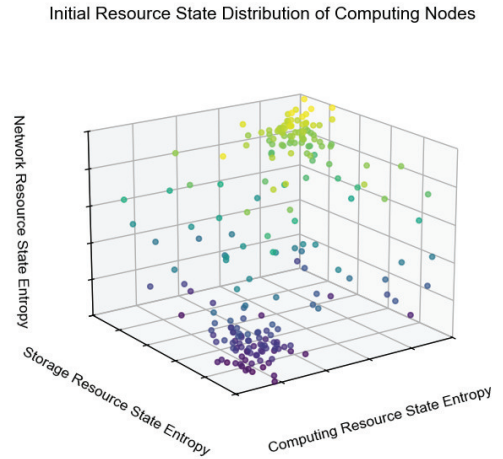


Figure 2. Initial resource state distribution of computing nodes.

creases node-state fluctuations, leading to high local entropy and global imbalance.

To achieve entropy balance, the scheduling strategy is implemented in two steps.

First, the scheduler maintains the stability of individual node entropy in the resource space R . It calculates the variance between task $t_i(c_i, l_i, s_i)$ and each candidate node $r_j(c_j, l_j, s_j)$, and selects the node $r_{\min}$ with the minimum variance for task matching and scheduling. This selection minimizes the state fluctuation caused by task allocation.

$$\begin{aligned} r_{\min} &= \min [D(t_i, R)] \\ &= \min_{j=1,2,\dots,N} [E(t_i - r_j)^2]. \end{aligned} \quad (6)$$

Second, the scheduler maintains the overall entropy balance of the resource space R . When W candidate nodes have similar minimum variances within a predefined interval, the node with the maximum variance from the balanced entropy is prioritized for selection. For example, if the balanced entropy is set to 0.469, the selection rule is expressed as follows:

$$\begin{aligned} r_{\min} &= \min [D(H(r_{j_{\min}}), 0.469)] \\ &= \min_{j=1,2,\dots,W} [E(H(r_{j_{\min}}) - r_j)^2]. \end{aligned} \quad (7)$$

If the selected node $r_{\min}$ belongs to service provider A, the task is scheduled across regions within A's resource pool. If the node belongs to another computing service provider B, interconnection between the resources of A and B is established, and the task is

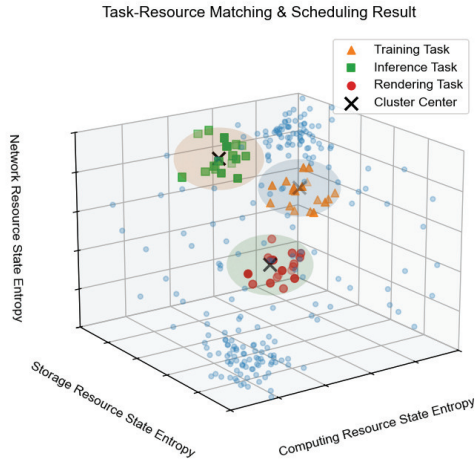


Figure 3. Resource distribution after task matching and scheduling.

assigned to B through cross-regional or cross-entropy scheduling.

If the task state deviates substantially from both the available resource states and the balanced entropy, new computing nodes may be planned to satisfy its requirements.

Figure 3 shows the resource-state distribution after applying the proposed matching and scheduling mechanism to the initial state in Figure 2. The three marker types represent training, inference, and rendering tasks, each forming a concentrated region in the three-dimensional resource-state space.

Based on the entropy distributions in Tables 1, 2, and 3, the system matches resources to task-specific requirements. Nodes assigned to the same task type occupy nearby positions in the entropy space, maintaining both individual-node stability and network-wide entropy balance.

3.6 Network Capacity Model

Resource matching in the IoC requires joint consideration of computing-resource availability and inter-node communication performance. Scheduling solely according to computing load may overlook the effects of network bandwidth utilization and communication latency on data synchronization. To address this limitation, this paper proposes a network capacity model that combines effective remote bandwidth with the entropy balance mechanism. The model quantifies the service capability of long-distance communication links and

supports task allocation in heterogeneous and dynamic environments.

The effective remote bandwidth is expressed as follows:

$$C_{net} = \text{Effective Remote Bandwidth} = \frac{D}{T_{\text{remote}} - N_{LD} \cdot RTT} \times \frac{1}{\eta}, \quad (8)$$

where

- C_{net} denotes the network capacity model;
- D denotes the volume of data to be transmitted;
- T_{remote} denotes the observed end-to-end time required to complete the data transmission;
- N_{LD} denotes the number of long-distance communications;
- RTT denotes the round-trip time of the link;
- η ($0 < \eta \leq 1$) denotes the bandwidth utilization factor.

Network capacity represents the effective data rate after fixed communication latency is excluded. It captures the combined effects of physical bandwidth, protocol efficiency, link reliability, and concurrent-task interference, thereby dynamically reflecting the actual service capability of RDMA dedicated lines in multi-tenant and multi-task environments.

The utilization factor η is not constant and varies with protocol overhead, congestion control, packet loss, and dynamic interference from coexisting traffic. In particular, even when RDMA dedicated lines are used and the physical links are dedicated to high-performance computing traffic, multiple parallel jobs may still share the underlying bandwidth. This sharing causes bandwidth convergence and intra-channel interference among concurrent jobs, which can be analogized to noise in communication. Although such interference does not compromise data integrity, it can substantially reduce the effective throughput of the target flow. Consequently, the actual single-flow bandwidth may remain below the physical upper limit and vary with the system load. Therefore, η should be monitored in real time to reflect changes in the operating conditions of RDMA dedicated lines.

During task matching, the scheduler uses historical or real-time measurements of C_{net} to determine whether the communication links between candidate nodes satisfy the data synchronization requirements of a task. For gradient-synchronization-intensive distributed training, the scheduler avoids node pairs whose

C_{net} is below the required threshold. Such paths are excluded even when computing resources are available, preventing communication from becoming a performance bottleneck. The model supports resource matching in large-scale, cross-domain, and high-concurrency computing scenarios.

IV. ARCHITECTURE OF THE INTERNET OF COMPUTE

4.1 System Architecture

In the traditional Internet architecture, the bearer network and service network are loosely coupled, as shown in Figure 4. The bearer network is responsible for packet forwarding and transmission, without separating the control plane from the transport plane, while the service network is deployed above it to support Over-The-Top (OTT) applications [25]. Because the bearer network cannot perceive upper-layer application states, it cannot effectively coordinate computing resources with network conditions. This limitation becomes increasingly evident in scenarios such as AI training and inference, which require close coordination between computing and network resources.

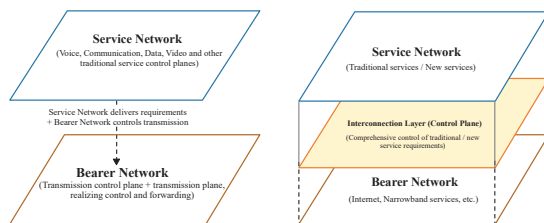


Figure 4. Comparison between the IoC and traditional Internet architectures.

Building on the previously proposed IoC architecture, this paper adopts a three-tier system comprising the Compute and Network Infrastructure Layer, Interconnection Resource Layer, and Application Service Layer. The architecture integrates computing services with the bearer network, as shown in Figure 5. Its key enhancement is the Interconnection Resource Layer, which separates the control plane from the bearer network and integrates network control with computing-service control. This layer enables unified abstraction and perception of computing and network resources, together with network-wide task–resource matching and scheduling.

4.1.1 Networking Architecture

As shown in Figure 5, the networking architecture comprises the Compute and Network Infrastructure Layer, Interconnection Resource Layer, and Application Service Layer. This layered design organizes system functions according to service requirements and improves architectural adaptability.

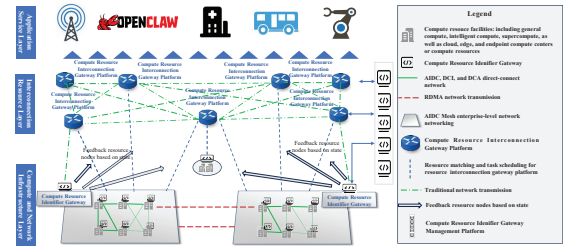


Figure 5. Three-tier architecture of the Internet of compute.

4.1.2 Protocol Architecture

The IoC protocol architecture builds on the five-layer Internet model to maintain compatibility with the existing Internet architecture. At the transport layer, it enhances high-performance transmission through technologies such as Remote Direct Memory Access (RDMA). At the application layer, it extends computing-task data protocols and computing identifiers to support task identification and resource-aware transmission over RDMA, as shown in Figure 6.

Computing Power Internet Networking Architecture	Computing Power Internet Architecture	Function	Computing Power Internet Protocol Suite	Internet Protocol Suite
Application Service Layer	Application Layer	Sense computing power identification, obtain available computing power resources, send task information, encapsulate task data, initiate data transfer.	TCP/IP Application Layer Protocols: HTTP/TFTP/DNS	HTTP/TFTP/DNS/Telnet/SMTP
Resource Interconnection Layer	Transport Layer	Implement data packet transmission control	RDMA/RDMAP	TCP/UDP
Compute-Network Infrastructure Layer	Network Layer	Implement high-speed transmission and routing of data packets between networks	IP(IPv4/IPv6 Addressing)	IP(IPv4/IPv6 Addressing)
	Data Link Layer	Identify unique host identifiers, provide error detection and correction	Ethernet(MAC Address Addressing)	Ethernet(MAC Address Addressing)
	Physical Layer	Implement data transmission on physical media	10G BASE-T/ Optical Fiber Ethernet	10G BASE-T/ Optical Fiber Ethernet

Figure 6. Protocol architecture of the Internet of compute.

4.2 Compute and Network Infrastructure Layer

4.2.1 Optical Bearer Network

The Compute and Network Infrastructure Layer provides the physical resources required for computation

and data transmission. Computing infrastructure includes general-purpose, intelligent, and supercomputing resources distributed across data centers, cloud platforms, and edge environments. Transmission infrastructure interconnects these resources through enterprise-level data-center connections [26], mesh networks [27], and the traditional Internet. Optical bearer networks provide the high-capacity transmission foundation for these interconnection modes.

4.2.2 Compute Cloud Private Network

The Compute Cloud Private Network provides high-bandwidth, low-latency, and highly reliable interconnection among computing resources. It is typically constructed using Optical Transport Network/Wavelength Division Multiplexing (OTN/WDM), core routing and switching, and Data Center Interconnection (DCI). These technologies connect geographically distributed data centers, computing clusters, and edge nodes, supporting cross-domain resource sharing and elastic scheduling. Technologies such as SRv6, IPv6+, and network slicing further improve resource-orchestration precision and service determinism while enabling integration with cloud-network platforms and edge-computing systems. Together, these capabilities support integrated computing-resource scheduling and the development of a sustainable computing and network ecosystem.

4.3 Interconnection Resource Layer

The Interconnection Resource Layer enables unified orchestration and scheduling of computing, storage, and network resources. It consists of three main components: the Computing Resource Identifier (CRID) Gateway, the Resource-Defined Memory Access Protocol (RDMA), and the entropy-balanced scheduling algorithm. By deploying CRID gateways, this layer enables network-wide resource-state perception. The resource state of node i is represented as

$$r_i = r_i(c_i, l_i, s_i), \quad (9)$$

where c_i , l_i , and s_i denote the computing, network communication, and storage states of the node, respectively.

Based on the perceived resource states, the Interconnection Resource Layer provides interfaces for resource identification, resolution, retrieval, query, and

path-scheduling parameters, supporting the on-demand use of heterogeneous resources by industries and enterprises. The scheduling algorithm then matches computing tasks with appropriate resources.

4.3.1 The Overall Architecture of Computing Resource Identifier

A unified identifier system for the IoC must extend beyond traditional static and flat addressing to represent the spatiotemporal attributes and dynamic states of computing resources. To this end, this paper proposes the Computing Resource Identifier (CRID), which supports five-level positioning and addressing across enterprise, city, computing center, server, and chip levels, as shown in Figure 7.

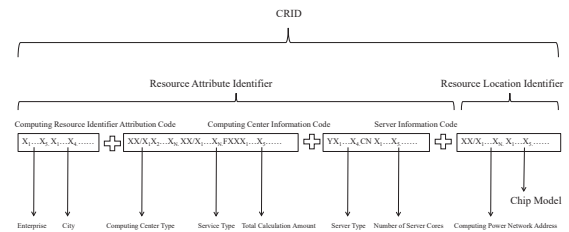


Figure 7. Overall architecture of the CRID.

The CRID Gateway collects and calculates the computing utilization c_i , storage utilization s_i , and inter-node communication utilization l_i of each node, thereby forming the resource state $r_i(c_i, l_i, s_i)$. These measurements provide a dynamic data basis for global resource scheduling, task optimization, and system coordination.

The computing identifier adopts a two-segment prefix-suffix encoding structure to describe resource attributes, scale, specifications, and paths. Combined with hierarchical resolution and dynamic verification, it enables the precise positioning and trusted authentication of distributed heterogeneous computing resources. CRID assigns each computing resource a unique identifier and supports resource description, positioning, addressing, and utilization-state perception.

As the core component of the CRID system, the gateway collects, verifies, resolves, and queries resource identifiers. It also generates unique task identifiers to support intelligent “task-resource” matching and entropy-balanced scheduling. At the data-collection level, the gateway perceives and evaluates three categories of node resources:

- **Computing resources:** The gateway collects the computing load, chip type (e.g., CPU, GPU, and TPU), chip model, chip count, core count, and memory capacity, providing the basis for calculating c_i ;
- **Storage resources:** The gateway collects the storage architecture (e.g., distributed or centralized) and storage capacity, providing the basis for calculating s_i ;
- **Network resources:** The gateway collects real-time information on network type, bandwidth capacity, and latency, and continuously monitors link stability and communication efficiency, providing the basis for calculating l_i .

Based on the collected resource information, the gateway dynamically evaluates the computing, network, storage, and comprehensive state entropy of each node. The comprehensive state entropy is expressed as

$$H(r_i) = F(H(c_i), H(l_i), H(s_i)), \quad (10)$$

where $F(\cdot)$ denotes the entropy fusion function for computing, network, and storage resources.

The result with the minimum entropy fluctuation is fed back to the entropy calculation module when the next task arrives. Through this feedback reinforcement mechanism, both the accuracy and environmental adaptability of the entropy-balanced scheduling algorithm can be continuously improved.

4.3.2 High-Performance Transmission Protocol

Traditional computing networks face decentralized resource scheduling, inefficient resource access, and limited traffic-control precision. To address these limitations, this paper proposes the Resource-Defined Memory Access Protocol (RDMAP), a memory-access protocol framework extended from RDMA over Converged Ethernet v2 (RoCEv2). By deeply integrating computing resource identifiers, RDMAP enables fine-grained perception and precise regulation of cross-domain traffic. It improves the computing efficiency ratio through coordinated resource scheduling and traffic management, promotes entropy balance within the network system, and enhances overall stability and resource utilization.

RoCEv2 provides high-performance, zero-copy data transmission. Existing approaches extend the reserved

bits of the Base Transport Header (BTH) to carry additional path or scheduling information. Conweave, proposed by the National University of Singapore in 2023, uses several BTH reserved bits as PathID/PathTag fields to identify the uplink path assigned to each packet. This design enables explicit and recoverable path selection, reducing receiver-side reordering overhead and improving link utilization.

SeqBalance, proposed by Huimin Luo et al. in 2024, uses the BTH reserved bits to perform sequence- or tag-based traffic balancing and support rapid forwarding decisions at network devices. It distributes packets across multiple paths while preserving in-order semantics or minimizing packet reordering.

Inter-node communication in the IoC requires both efficient data transmission and awareness of the remote computing state, supporting the “Compute as Address” paradigm. RDMAP therefore extends the protocol semantics of RoCEv2 while maintaining compatibility with its basic transmission mechanism. This extension gives computing resources global addressing and dynamic binding capabilities, allowing existing RDMA devices to enable compute-aware transmission through lightweight upgrades.

RDMAP adopts a four-tier architecture comprising the Computing Orchestration Layer (COL), Transport Extension Layer (TEL), Resource Identification Layer (RIL), and Memory Semantics Layer (MSL), as shown in Figure 8.

In terms of frame structure, RDMAP retains the basic RoCEv2 format and introduces lightweight extensions to the BTH reserved bits. One bit is used as the RDMAP flag, where 0 indicates a standard RoCEv2 frame and 1 indicates an RDMAP frame. A new RDMAP Header is inserted before the payload. The header contains a 1-byte start and end identifier and the Computing Resource Identifier (CRID), embedding computing information into data packets to support long-distance computing-network collaboration, as shown in Figure 9.

4.3.3 Scheduling Algorithm

The computing-resource scheduling algorithm matches different types of computing tasks with optimal resources. It focuses on two representative workloads: training and inference. Their distinct computing characteristics and scheduling objectives require differen-

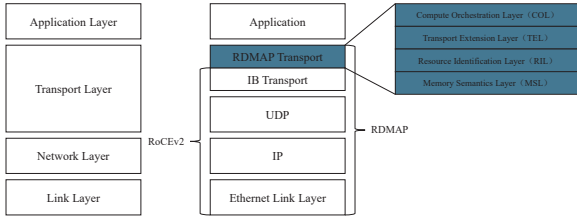


Figure 8. RDMA protocol architecture.

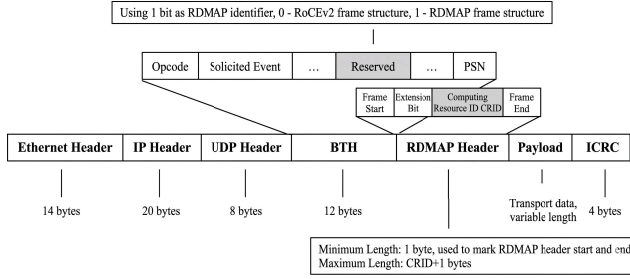


Figure 9. RDMA frame structure.

tiated strategies, providing the basis for subsequent scheduling models and algorithms.

As artificial intelligence models continue to grow, the computing requirements of training and inference tasks increase accordingly. A single node or local data center may struggle to simultaneously provide elastic resources, low-latency responses, and energy-efficient operation.

Training tasks are compute-intensive and long-running, with strong requirements for GPU performance and energy efficiency. Inappropriate resource allocation may cause prolonged node overload and reduced execution efficiency. Inference tasks are short-lived and highly concurrent, requiring low latency and rapid, elastic resource allocation.

Computing-resource scheduling is therefore a joint optimization problem involving heterogeneous tasks [28] and distributed computing, storage, and network resources [29]. Its objective is to balance task latency, energy constraints, and resource utilization [30]. Unlike traditional network routing, computing scheduling emphasizes joint decision-making across multiple resource dimensions and elastic resource orchestration.

The scheduling process consists of two steps.

1) State Types and State Entropy Parameter Settings.

The algorithm decomposes each task, analyzes the computing, network, and storage requirements of its subtasks, and evaluates the states of available computing nodes.

Based on the resource state entropy model, the algorithm considers three scheduling factors: computing state c , communication state l , and storage state s . These factors represent the available computing capability, inter-node communication condition, and storage capacity for caching or intermediate results, respectively.

2) Weight Setting.

Based on the global state entropy and entropy balance model, the comprehensive state entropy of resource node r_i is expressed as

$$H(r_i) = w_c H(c_i) + w_l H(l_i) + w_s H(s_i), \quad (11)$$

where w_c , w_l , and w_s denote the weights of computing, network, and storage resources, respectively, and satisfy $w_c + w_l + w_s = 1$. The weights can be dynamically adjusted according to scenario requirements.

V. APPLICATION SCENARIOS OF THE INTERNET OF COMPUTE

5.1 Large Model Training

In the IoC scenario, large model training can utilize cross-domain heterogeneous resources to achieve efficient parallel computing. As shown in Figure 10, the cross-domain heterogeneous training architecture consists of three layers: the Application Service Layer, Interconnection Resource Layer, and Computing-Network Infrastructure Layer.

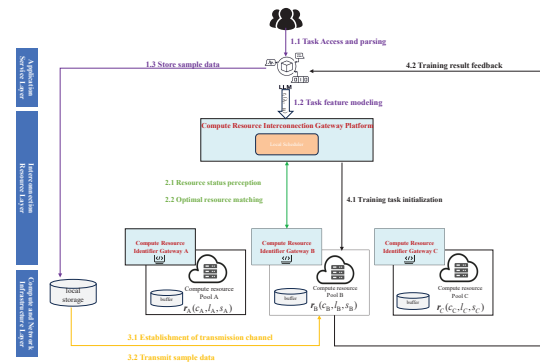


Figure 10. Model training process in the Internet of compute.

5.1.1 Training Architecture

First, the Application Service Layer describes training tasks, obtains the task parameters $t_i(\lambda_i, \mu_i)$, and

submits task requests to the Interconnection Resource Layer. The task description includes computing, storage, and network parameters.

- Computing parameters: Training sample size, model parameter size, number of training epochs, batch size, training/validation/testing ratios, and duration per training epoch, i.e., service time μ_i .
- Storage parameters: Total cached sample data volume and incremental sample data volume per arrival.
- Network parameters: Incremental sample volume per arrival, i.e., the transmitted sample data volume, computing network capacity C_{net} , and arrival-time distribution λ_i .

Second, the Interconnection Resource Layer receives task requests, obtains the task parameters $t_i(\lambda_i, \mu_i)$, uniformly encapsulates the tasks, and provides task-scheduling capabilities. By perceiving the underlying resource states $r_i(c_i, l_i, s_i)$ through the CRID Gateway, this layer matches resource states with task requirements.

- Computing Identifier Gateway: Collects computing, storage, and network parameters and continuously monitors changes in the state $r_i(c_i, l_i, s_i)$ of the selected resource nodes.
- RDMA Transmission Control: After the CRID Gateway filters the computing nodes, RDMA provides long-distance lossless transmission based on the RDMA protocol. It adjusts the transmission rate, congestion window, and traffic priority in real time according to the network capacity model and network resource state entropy, thereby ensuring data synchronization during large model training.
- Scheduling Algorithm: Matches the task parameters $t_i(\lambda_i, \mu_i)$ with the node state $r_i(c_i, l_i, s_i)$ and accurately assigns training tasks to optimal computing nodes. It dynamically adjusts resource allocation according to real-time node states $r_i(c_i, l_i, s_i)$, thereby maintaining balanced resource utilization during training.

Third, the Computing-Network Infrastructure Layer provides the physical infrastructure for task execution and data transmission. It provides the computing, storage, and network capabilities $r_i(c_i, l_i, s_i)$ of interconnected heterogeneous resource nodes and synchronizes their capabilities and states with the Interconnection Resource Layer in real time.

- Computing Resources: The core computing re-

sources for distributed large model training consist of heterogeneous intelligent computing nodes optimized for AI training scenarios.

- Storage Resources: Local node storage and remote resource-pool storage provide sample data and model parameters for large model training.
- Network Resources: RDMA-based channels provide high-speed, lossless remote data transmission for training tasks.

5.1.2 Training Process

The large model training process in the Internet of Compute scenario consists of four core stages: training task identification, task-compute matching, training data transmission flow, and training task execution. The workflow of each stage is described below.

Training Task Identification.

- 1) Task Access and Parsing: The Application Service Layer receives user-initiated large model training tasks, parses core parameters including training sample size, model parameter size, training epochs, and batch size, and extracts the corresponding computing, storage, and network requirements.
- 2) Task Feature Modeling: Based on the parsed task parameters, the Application Service Layer performs statistical modeling of the training tasks and generates the task features $t_i(\lambda_i, \mu_i)$. The task requests and feature vectors are then sent to the Gateway Platform of the Interconnection Resource Layer.
- 3) Sample Data Storage: The Application Service Layer stores the training sample data in the local storage resource pool, completing data preparation before training execution.

Task-Compute Matching.

- 1) Resource State Perception: The Computing Identifier Gateway of the Interconnection Resource Layer collects the real-time computing, storage, and network states of the three types of resource nodes (A, B, and C) in the Computing-Network Infrastructure Layer and synchronizes the state information with the Gateway Platform.
- 2) Optimal Resource Matching: According to the task requirements and underlying resource states, the Gateway Platform applies the entropy-balanced scheduling algorithm to select the optimal node, such as Node C, from Nodes A, B, and C.

Training Data Transmission Flow.

- 1) **Transmission Channel Establishment:** The Gateway Platform issues training-execution instructions to the selected target computing node and completes the standardized encapsulation and distribution of the training task. Meanwhile, the Interconnection Resource Layer establishes an end-to-end high-speed, lossless data-transmission channel between the local storage resource pool and the target computing node using the RDMAP protocol.
- 2) **Sample Data Transmission:** The local storage resource pool transmits the pre-trained model weight files and training sample data to the target computing node, completing data-resource preparation before training execution.

Training Task Execution.

- 1) **Training Task Startup:** After receiving the training instructions, the target computing node initializes the training environment, prefetches and loads training-sample batches through the RDMAP high-speed channel, and starts the training task.
- 2) **Training Result Return:** Upon completing model training, the selected resource node transmits the final trained model parameters and results back to the Application Service Layer through the network.

5.2 Large Model Inference

5.2.1 Inference Architecture

In the Internet of Compute scenario, the demand for concurrent large model inference has increased significantly. Multiple inference tasks can be processed by offloading them across heterogeneous resources in different domains, as shown in Figure 11.

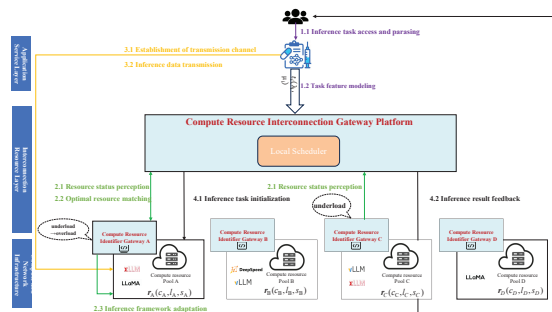


Figure 11. Model inference process in the IoC.

First, the Application Service Layer serves inference scenarios such as intelligent question answering and multimodal generation. It parses and describes inference tasks, obtains the task parameters $t_i(\lambda_i, \mu_i)$, and sends the resulting inference requests to the Interconnection Resource Layer.

- **Computing parameters:** Input prompt length, expected generated-content length, and model parameter scale.
- **Storage parameters:** Required model-parameter cache volume, selected-model weight size, and KV cache occupancy.
- **Network parameters:** Arrival-time distribution λ_i , transmitted data volume per request, and network bandwidth and latency requirements.

Second, the Interconnection Resource Layer uses the Computing Identifier Gateway to perceive the underlying resource states $r_i(c_i, l_i, s_i)$. Based on these states and the inference task $t_i(\lambda_i, \mu_i)$, model type, and underlying chip model, the layer selects resource nodes with optimal resource utilization and matches the optimal inference acceleration framework. The inference tasks are then distributed to the matched resource nodes to maintain balanced resource states and improve inference efficiency.

- **Computing Identifier Gateway:** Collects and updates the computing, storage, and network parameters of each node in real time, thereby continuously perceiving its resource state $r_i(c_i, l_i, s_i)$.
- **Scheduling Algorithm:** Evaluates candidate resource states according to the latency and concurrency requirements of inference tasks and the task parameters $t_i(\lambda_i, \mu_i)$, and continuously selects the optimal resource nodes for different inference tasks.

Third, the Computing-Network Infrastructure Layer provides computing resources, models, and inference acceleration frameworks across interconnected heterogeneous resource pools. The Computing Identifier Gateways deployed in these resource pools monitor their computing, storage, and network communication states $r_i(c_i, l_i, s_i)$ in real time and synchronize the results with the Interconnection Resource Layer through interfaces. Based on the task-model matching results, the layer activates the deployed inference acceleration frameworks on the selected resource nodes to provide optimal inference acceleration.

- **Computing Resources:** Comprise heterogeneous

intelligent computing nodes deeply optimized for AI scenarios and serve as the core computing carriers for distributed large model inference.

- **Inference Acceleration Frameworks:** Multiple mainstream inference acceleration frameworks are pre-deployed to support different chip architectures and model types.
- **Storage Resources:** Distributed storage resource pools provide high-speed caching and storage for inference model parameters, hot request samples, and temporary inference results.
- **Network Resources:** Provide low-jitter and highly reliable transmission for cross-node inference task distribution, model weight synchronization, and inference result return.

5.2.2 Inference Process

The large model inference process in the IoC consists of four core stages: inference task identification, task-compute matching, inference data transmission flow, and inference task execution.

Inference Task Identification.

- 1) **Inference Task Access and Parsing:** The Application Service Layer receives inference requests and parses their core parameters.
- 2) **Task Feature Modeling:** Based on task parameters such as the task arrival characteristic λ_i and service time characteristic μ_i , the Application Service Layer models each task to generate the task feature $t_i(\lambda_i, \mu_i)$ and sends the generated feature to the Gateway Platform of the Interconnection Resource Layer.

Task-Compute Matching.

- 1) **Resource State Perception:** Through the Computing Identifier Gateway deployed on each node, the Interconnection Resource Layer monitors the computing, network, and storage resource states and the deployed inference acceleration frameworks of all candidate resource nodes in the Computing-Network Infrastructure Layer in real time. Core parameters, including real-time node load, available computing capability, and link bandwidth, are collected and synchronized with the Gateway Platform.
- 2) **Optimal Resource Matching:** The Gateway Platform runs the entropy-balanced scheduling algorithm and jointly considers task requirements and

node resource states. It selects the optimal inference node from the cross-domain candidate resource pools according to resource-state stability and task-resource compatibility, and then issues the inference execution instructions.

- 3) **Inference Framework Adaptation:** According to the model type and parameter scale of the inference task, as well as the chip architecture of the target node, the appropriate inference acceleration framework is automatically matched, and the corresponding inference instructions are then issued.

Inference Data Transmission Flow.

- 1) **Transmission Channel Establishment:** Based on the global network resource state perceived by the Computing Identifier Gateway, the Interconnection Resource Layer plans the optimal transmission path and establishes an end-to-end low-latency dedicated transmission channel between the Application Service Layer and the target inference node.
- 2) **Inference Data Transmission:** Through the dedicated transmission channel, inference requests are distributed in the uplink direction, while dependent data, including model weights and the KV cache, is pulled on demand.

Inference Task Execution.

- 1) **Inference Task Startup:** After receiving the scheduling instructions, the target inference node initializes the inference environment, loads the corresponding model weights, activates the matched inference acceleration framework, and executes the inference process.
- 2) **Return Inference Results:** After completing the assigned inference service tasks, the computing resource node returns the inference results to the Application Service Layer.

Throughout the inference service, the Computing Identifier Gateway continuously collects the real-time load, resource utilization, and state entropy changes of the target inference node and reports the runtime data to the Gateway Platform. When the node load exceeds the preset threshold or its state entropy rises significantly, new inference requests are dynamically offloaded to other stable and suitable inference nodes.

VI. EXPERIMENTAL SIMULATION

To fully verify the feasibility and practical value of the Internet of Compute (IoC) architecture proposed in this paper, this study carried out physical deployment and systematic tests in a real heterogeneous IoC environment.

6.1 Experiment on Cross-Domain Training of Large Models Based on the IoC

This section investigates distributed large model training and evaluates the combined performance of the entropy balance theory, the RDMAP protocol, and the entropy-balanced scheduling algorithm proposed in this paper. The experiments assess resource efficiency and system stability through a fair comparison with a representative baseline, thereby evaluating the technical advantages of the IoC architecture in cross-domain training scenarios.

6.1.1 Experimental Design

Experimental Environment Configuration.

A cross-domain training validation environment is constructed based on the three-tier IoC architecture. Full-stack deployment and parameter configuration are performed across the Application Service Layer, Interconnection Resource Layer, and Compute and Network Infrastructure Layer. This configuration coordinates the training workload, scheduling mechanism, and underlying hardware infrastructure, enabling a comprehensive evaluation of the architecture in large model training scenarios.

1) Application Service Layer

The Application Service Layer serves as the unified entry point for cross-domain training tasks and is responsible for task admission and parsing, feature modeling, and resource demand delivery. This experiment considers parameter-efficient fine-tuning of a large language model as the primary test scenario. A standardized training workload is constructed using an open-source model and a public dataset, providing a controllable benchmark for subsequent resource scheduling and performance evaluation.

The open-source Qwen3-8B large language model is selected as the training workload and fine-tuned using Low-Rank Adaptation (LoRA) with FP16 precision. The SoulChatCorpus open-source mental health dataset

contains 10 000 samples and is divided into training, validation, and test sets at a ratio of 6:2:2. After parsing the task parameters, the Application Service Layer statistically models the training task according to the computing task model described in Section 3.3. It generates a task feature vector containing the arrival rate λ and service time μ and delivers the vector to the Interconnection Resource Layer. The workload exhibits low-frequency and long-duration characteristics. The service time of an individual task follows the normal distribution $\mathcal{N}(25 \text{ min}, 2 \text{ min})$, and the task arrival rate is $\lambda = 0.2$ tasks per hour.

The core training parameters are summarized below.

- Number of training epochs: 10;
- Per-GPU batch size: `batch_size = 32`;
- Gradient accumulation steps: 4;
- Distributed training strategy: Data Parallelism;
- Gradient synchronization mechanism: Global AllReduce;
- Data prefetching mechanism: The local cache capacity is set to 500 samples, and the prefetch threshold is set to 200 samples. Remote data fetching is automatically triggered when the number of locally cached samples falls below the threshold.

2) Interconnection Resource Layer

The Interconnection Resource Layer implements the entropy balance theory and enables unified orchestration and scheduling of computing and network resources. It comprises three collaboratively operated components: the Computing Resource Identifier Gateway, the RDMAP high-performance transmission protocol, and the entropy-balanced scheduling algorithm.

- Computing Resource Identifier Gateway:
Based on the five-level positioning and addressing framework of Enterprise–City–Computing Center–Server–Chip, the Computing Resource Identifier Gateway collects the computing load, storage occupancy, and network link status of each node at an interval of 1 min. According to the information entropy formula, the gateway automatically calculates the computing, network, and storage resource state entropy of each node, providing a quantitative basis for state evaluation and scheduling decisions. The statistical sampling interval for resource state entropy is also set to 1 min. Thus, ten uniformly distributed valid samples are collected during each training cycle, covering its complete life cycle.
- RDMAP Transmission Protocol:

RDMAP is implemented as an extension of RoCEv2 and deeply integrates the Computing Resource Identifier (CRID) to support compute-aware, long-distance, and lossless transmission. In conjunction with the network capacity model described in Section 3.6 and the network resource state entropy, RDMAP dynamically adjusts the transmission rate, congestion window, and traffic priority, ensuring stable and efficient gradient synchronization and data fetching during large model training. The nominal link bandwidth is 100 Gbps. RDMAP is fully compatible with standard RoCEv2 devices and supports lightweight upgrades of existing networks.

- **Entropy-Balanced Scheduling Algorithm:** The entropy-balanced scheduling algorithm takes the task feature vector and node resource state entropy as inputs and uses global entropy balance as the optimization objective. It adopts a two-stage strategy consisting of minimum state-variance matching and global balanced scheduling to select computing nodes with the most stable comprehensive resource states. Considering the compute-intensive and communication-sustained characteristics of the training workload, the default weights of computing, network, and storage resources are set to $w_c = 0.5$, $w_l = 0.3$, and $w_s = 0.2$, respectively.

3) Compute and Network Infrastructure Layer

The Compute and Network Infrastructure Layer provides the physical hardware required for cross-domain training, including computing, storage, and transmission resources. The experimental environment consists of one storage resource node and three heterogeneous computing nodes interconnected through a high-speed network. Different proportions of background load are assigned to the initial states of the computing nodes to emulate resource occupation and dynamic variations in practical production environments.

The hardware configurations of the computing nodes are presented in Table 4. All nodes are configured with the same versions of the operating system, deep learning training framework, and dependent libraries to ensure hardware comparability. The independent storage node stores the training dataset and pre-trained model weights and provides high-throughput read and write services for sample data and model files. The underlying network uses high-speed Ethernet. A pro-

fessional network impairment emulator reproduces a cross-domain wide-area link with an equivalent physical distance of 100 km, a baseline round-trip time of 1.2 ms, default latency jitter of ± 0.3 ms, and a baseline packet loss rate of 1×10^{-4} . Periodic background traffic is injected to emulate wide-area link congestion. The congestion cycle is 5 min, each event lasts 30 s, and the available link bandwidth is reduced by 40% during congestion. This configuration evaluates the architecture under fluctuating network conditions.

Baseline Comparison Method.

To fairly evaluate the performance advantages of the proposed approach, a baseline combining TCP/IP transmission with random scheduling, denoted as TCP+Random, is selected for comparison.

- **Baseline: TCP/IP + Random Scheduling (TCP+Random).** The baseline uses the conventional TCP protocol for data transmission and randomly assigns training tasks to available computing nodes. It represents the training mode under the traditional Internet architecture and serves as the control group.
- **Proposed Scheme.** The proposed scheme employs RDMAP for compute-aware transmission and combines it with the entropy-balanced scheduling algorithm based on resource state entropy, thereby implementing the complete capabilities of the three-tier IoC architecture.

Evaluation Metrics.

The experiments employ quantitative metrics from two perspectives: resource efficiency and system stability.

- **Resource Efficiency Metrics**
 - **Average Computing Resource Utilization:** The average GPU utilization during the training period, reflecting the reuse level of computing resources.
 - **Average Network Resource Utilization:** The average link bandwidth utilization during the training period, reflecting the carrying efficiency of network resources.
- **System Stability Metric**
 - **Comprehensive Resource State Entropy:** The weighted sum of the computing, network, and storage resource state entropy of a node, reflecting the degree of fluctuation in the overall system state. A lower entropy value indicates a more stable operating state.

Table 4. Resource allocation for heterogeneous computing nodes.

Node Type	GPU	GPU Memory (GB)	Memory (GB)	Network Bandwidth (Gbps)	Storage (GB)	Load Rate
Node-X	A100×4	40	512	100	2048	10%
Node-Y	A100×4	40	512	100	2048	50%
Node-M	A100×3	40	384	50	1024	60%

6.1.2 Experimental Procedure

The experiment is conducted in a heterogeneous IoC simulation environment using a controlled-variable approach to ensure comparability between the two groups. The procedure is described below.

Experimental Environment Deployment and Initialization.

- 1) Physical node and simulation environment setup: One storage node and three computing nodes are deployed according to the parameters of the Compute and Network Infrastructure Layer. The same operating system, training framework, and required libraries are installed on all nodes. The simulation platform is configured with the baseline latency, jitter, packet loss rate, and periodic congestion events of the 100 km wide-area link.
- 2) Dataset and model initialization: The SoulChat-Corpus dataset is preprocessed, partitioned, and stored on the storage node, and the pre-trained Qwen3-8B model is deployed.

Control Group Configuration and Execution.

- 1) Control group configuration: The control group uses the standard TCP/IP protocol and a random node scheduling strategy. Node assignments and data transmission modes remain fixed throughout training, with no dynamic adjustment.
- 2) Control group training: The training task performs model fine-tuning for 10 min. Computing, network, and storage resource utilization is collected at 1-min intervals. The sampling clock is triggered when training begins, and ten entropy samples are collected at 1-min intervals. Training interruptions, congestion anomalies, and other events are recorded simultaneously.

Experimental Group Configuration and Execution.

- 1) Experimental group configuration: RDMA is enabled at the network layer, Computing Resource Identifier Gateways are deployed on all nodes, real-time resource state collection is activated, and the entropy-balanced resource scheduling algorithm is

enabled.

- 2) Node matching and task delivery: The Application Service Layer models the training task. The Interconnection Resource Layer obtains the real-time node states and entropy values through the Computing Resource Identifier Gateway, selects the optimal node, Node-X, and issues the training instruction.
- 3) Experimental group training: After environment initialization, the selected node starts the training task. The gateway continuously collects node states at the configured interval of 100 ms and completes ten rounds of resource state entropy sampling, calculation, and recording at fixed 1-min intervals. Based on the node loads, the scheduling algorithm dynamically adjusts the data transmission strategy to stabilize the node load distribution until the 10-min training task is completed.

To reduce random errors, both the control and experimental groups undergo five complete trials. Before each trial, the initial node loads, network simulation environment, and model parameters are reset. After outlier removal, the data collected from the five trials are averaged to form the final statistical dataset, which is used to plot the resource state entropy distribution and resource utilization curves.

6.1.3 Experimental Results

To evaluate the proposed IoC architecture in cross-domain large model training, Qwen3-8B fine-tuning is used as the test workload, and two strictly controlled groups are compared. The results are analyzed in terms of resource utilization efficiency and resource state stability. Computing and network utilization quantify resource coordination and reuse, while comprehensive resource state entropy characterizes resource-load fluctuations and operating-state stability.

1) Analysis of Resource Utilization Results

The experiment uses 10 min of model training as the observation period and continuously collects the

Table 5. Comparison of computing and network resource utilization.

Time (min)	Without IoC Compute Util. (%)	With IoC Compute Util. (%)	Compute Util. Improvement (%)	Without IoC Network Util. (%)	With IoC Network Util. (%)	Network Util. Improvement (%)
1	48.76	68.70	40.89	38.62	50.69	31.25
2	53.16	68.01	27.93	38.74	51.05	31.78
3	51.79	67.70	30.72	37.16	50.87	36.89
4	49.00	64.08	30.78	35.89	49.41	37.67
5	49.82	64.54	29.55	37.93	50.77	33.85
6	52.45	66.67	27.11	38.65	51.11	32.24
7	53.36	66.46	24.55	37.93	50.92	34.25
8	50.92	67.96	33.46	35.52	49.62	39.70
9	50.10	67.21	34.15	36.94	50.84	37.63
10	49.75	65.68	32.02	38.19	51.12	33.86
Average	—	—	31.12	—	—	34.91

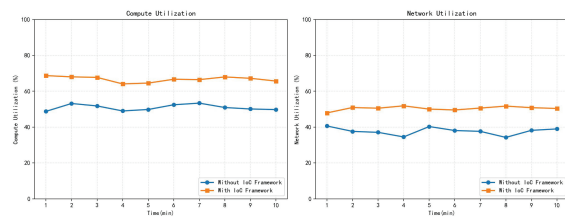


Figure 12. Comparison of computing and network resource utilization.

computing and network resource utilization of the target nodes in both groups. The results are presented in Table 5.

During the 10-min training period, computing resource utilization in the experimental group remains above 60%, while network utilization remains above 49%. Both metrics remain efficient and stable under periodic congestion. In the control group, computing utilization falls below 50% at its lowest point, and network utilization falls below 40% during congestion, with both exhibiting substantial fluctuations. Relative to the control group, the experimental group improves average computing utilization by 31.12% and average network utilization by 34.91%. The corresponding trends are shown in Figure 12.

These results indicate that the IoC architecture coordinates computing and network resources through millisecond-level resource state perception by the Computing Resource Identifier Gateway, adaptive traffic regulation by RDMA, and precise matching by the entropy-balanced scheduling algorithm. This coordination alleviates computing-network decoupling in conventional architectures and improves overall hardware resource utilization.

2) Analysis of Resource State Entropy Results

Table 6. Comparison of weighted comprehensive resource state entropy.

Time (min)	Without IoC Weighted Entropy	With IoC Weighted Entropy	Entropy Reduction (%)
1	0.487	0.217	55.4
2	0.401	0.188	53.1
3	0.437	0.194	55.6
4	0.444	0.191	57.0
5	0.396	0.183	53.8
6	0.385	0.176	54.3
7	0.362	0.183	49.4
8	0.423	0.188	55.6
9	0.415	0.194	53.3
10	0.439	0.199	54.7
Average Entropy Reduction			54.2

At 1-min intervals, the experiment collects the computing, network, and storage resource state entropy of the target nodes in both groups and calculates the weighted comprehensive entropy. The results are presented in Table 6.

The entropy samples of the experimental group are concentrated in the low-entropy region below 0.3, with no high-entropy outliers. In contrast, the control-group samples are dispersed over the medium-to-high entropy range of 0.3–0.6, with numerous samples concentrated in the higher-entropy region. Both the overall entropy level and distribution dispersion of the control group are substantially greater than those of the experimental group. On average, the weighted comprehensive entropy of the experimental group is 54.2% lower than that of the control group. The three-dimensional entropy distribution is shown in Figure 13.

These results validate the effectiveness of the entropy balance theory and scheduling algorithm. The proposed approach smooths load fluctuations, stabilizes node op-

erating states, and mitigates the poor system stability caused by the lack of resource-state awareness in conventional architectures.

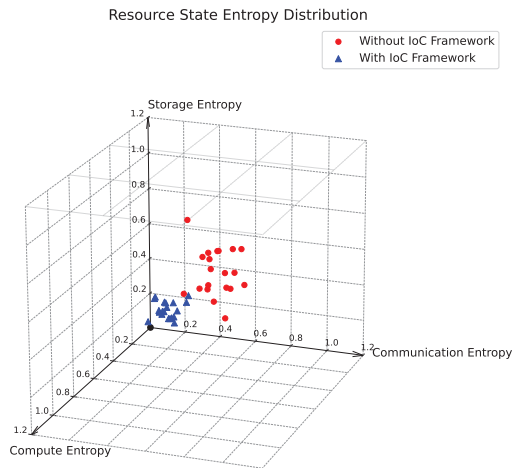


Figure 13. Distribution of resource node state entropy.

6.2 Experiment of Large Model Inference Scenario Based on the IoC

6.2.1 Experimental Design

Experimental Environment Configuration.

The experimental environment follows the three-layer architecture of the IoC and comprises the Application Service Layer, Interconnection Resource Layer, and Computing-Network Infrastructure Layer. Computing nodes that meet the requirements of large model inference are configured to comprehensively evaluate the architecture's optimization performance in inference scenarios.

Application Service Layer.

The Application Service Layer is responsible for initiating inference task requests. The performance requirements of inference services are determined based on differences in model scale, computing-resource demand, and response latency among inference tasks. This experiment uses real-time inference calls to the large language model DeepSeek-R1 on the requester side as the core scenario and designs typical inference task loads to provide a controllable benchmark for subsequent node resource scheduling and performance evaluation.

Interconnection Resource Layer.

The Interconnection Resource Layer serves as the core layer for implementing the entropy balance theory of the IoC and unified resource orchestration and scheduling through collaboration between the Computing Identifier Gateway and the entropy-balanced scheduling algorithm. Based on the five-level positioning and addressing framework Enterprise–City–Computing Center–Server–Chip, the Computing Identifier Gateway periodically collects node resource metrics. It then automatically calculates the computing, network, and storage entropy of each node using the resource state entropy formula, providing a quantitative basis for resource state evaluation. The entropy-balanced scheduling algorithm models inference demands using the task parameters μ and λ . Inference tasks are characterized by short-term bursts, high-frequency arrivals (high λ), and short service times (low μ). Using these task characteristics together with the resource state entropy reported by the Computing Identifier Gateway, the algorithm dynamically matches inference tasks to nodes with stable states, thereby achieving optimal alignment between task requirements and underlying computing resources.

Computing-Network Infrastructure Layer.

The Computing-Network Infrastructure Layer provides the physical computing and transmission resources required to support IoC operation. This experiment constructs an IoC simulation environment comprising 16 heterogeneous computing nodes interconnected through high-speed networks, with differentiated computing, storage, and network capabilities. Each node carries an initial load generated by various computing tasks, such as scientific computing, rendering and live streaming, and inference Q&A, and the load changes dynamically as these tasks are executed.

Inference Node Configuration.

All computing nodes selected in this experiment meet the minimum hardware standards required to run the DeepSeek-R1 model at full performance. During inference, both the Prefill and Decode stages use an 8-card tensor-parallel distributed architecture (Tensor Parallelism = 8). Mixed Tensor Precision (MTP) and Automatic Precision Control (APC) strategies are enabled, and FP8 quantized weights are loaded to maximize inference efficiency and resource utilization. The transmission infrastructure combines enterprise-level direct data center connections with a mesh communica-

tion network. This configuration supports interconnection and collaborative scheduling across geographically distributed data centers and provides high-bandwidth, low-latency transmission for distributed inference.

Experimental Setup.

Progressive experiments are designed along two dimensions: Resource State Stability and task performance. Resource state provides the basis for scheduling decisions, and its fluctuations directly affect the predictability of task execution. Improvements in resource stability should therefore be reflected in task performance gains. Accordingly, the experimental setup includes the following two experiments:

- Experiment 1: Stability experiment of node resource states under the IoC architecture. This experiment compares the computing entropy, network entropy, and storage entropy of nodes with and without the IoC architecture to verify the architecture's role in improving resource state stability.
- Experiment 2: Comparative performance experiment between traditional internet architecture and the IoC architecture. Comparison scenarios are established for deployments based on the traditional internet architecture and the IoC architecture to evaluate the actual performance improvements provided by the IoC architecture in inference scenarios.

Experimental Implementation.

A unified task load and model structure are used across different configurations to ensure comparability. Data are collected and analyzed over multiple rounds under high-concurrency conditions. Task loads are generated following standard benchmarking methods used by open-source large model inference frameworks, with two typical workload-generation methods:

- Fixed concurrency stress test: To represent resource-intensive and computing-intensive workloads, the maximum number of simultaneously active inference requests is limited, and the concurrency level is gradually increased in integer steps to evaluate the system's resource scheduling boundaries and stability.
- Dynamic request rate test: This test simulates the inference-request load characteristics of small- and medium-sized large language models in high-concurrency scenarios. A Poisson distribution is used to simulate user request flows, with request rates increased stepwise to detect throughput limits

and response latency variations.

To ensure the statistical representativeness of the experimental data and the adequacy of the comparative analysis, each test includes at least 1000 requests.

6.2.2 Experimental Procedure

Experiment 1: Stability Experiment of Node Resource States under the IoC Architecture.

This experiment aims to verify the impact of the IoC architecture on the stability of node resource states. A group of identically configured computing nodes is selected for the experiment. Under a consistent workload, inference tasks are executed under two conditions: with and without the IoC architecture. During the experiment, the Computing Identifier Gateway monitors node resource states in real time and calculates their entropy. The gateway periodically collects three categories of resource metrics:

- Computing resources: computing volume, chip series, chip model, number of chips, core count, and memory capacity;
- Network resources: network type, bandwidth, and network latency, together with continuous monitoring of network stability;
- Storage resources: storage architecture and storage capacity.

After data have been continuously collected for 24 hours, the hourly measurements are discretized into preset utilization intervals, and the probability distribution across these intervals is calculated. Based on these probability distributions, the Computing Identifier Gateway automatically generates the computing entropy, network entropy, and storage entropy of each node using the information entropy formula. The three entropy values are then compared between nodes under the two architectures to verify the role of the IoC architecture in improving resource state stability.

Experiment 2: Comparative Performance Experiment between Traditional Internet Architecture and the IoC Architecture.

This experiment focuses on verifying the impact of the IoC architecture on distributed-node inference relative to the traditional Internet architecture in terms of processing mode, network capacity, and physical distance. Three controlled groups are configured with equivalent computing resources.

- Group 1: Random centralized nodes in the tra-

ditional Internet architecture. This group adopts the traditional protocol stack without the Interconnection Resource Layer. All inference tasks are processed serially on the same computing node. The group has no computing identifier awareness or dynamic scheduling capability and serves as the baseline for traditional centralized inference.

- Group 2: Random distributed nodes in the traditional Internet architecture. This group adopts the traditional protocol stack without the Interconnection Resource Layer. Inference tasks are scheduled to multiple geographically distributed nodes for parallel processing. Inter-node network bandwidth is limited to simulate public network transmission. The group has no node resource state entropy perception or entropy-balanced scheduling mechanism.
- Group 3: Random distributed nodes in the Internet of Computing architecture. This group fully deploys the three-layer IoC architecture. The Computing Identifier Gateway perceives the computing entropy, network entropy, and storage entropy of each node in real time. The entropy-balanced scheduling algorithm dynamically matches inference tasks to nodes with the lowest comprehensive state entropy, thereby enabling cross-domain distributed parallel inference. This group evaluates the comprehensive performance advantages of the IoC architecture in distributed mode.

During inference, the inter-node transmission time T contributes to the initial data synchronization delay and affects the efficiency of sustained data exchange. It therefore directly determines the overall end-to-end inference latency. Accordingly, Time To First Token (TTFT) and Time Per Output Token (TPOT) are selected as dependent variables to characterize the qualitative relationship between transmission time and inference latency. To examine how different input-output lengths affect this relationship, three input-output length configurations are used: 1024–1024 tokens, 2048–2048 tokens, and 4096–4096 tokens.

6.2.3 Experimental Results

Experiment 1: Stability Experiment of Node Resource States under the IoC Architecture.

In inference scenarios, tasks are characterized by short-term bursts, sensitivity to latency, and rapid re-

source release. In this experiment, the network, computing, and storage resource state entropy values of target nodes in both groups are continuously collected on a per-epoch basis. Network latency plays a dominant role in inference tasks, computing resources exert a secondary influence during bursts, and storage has a negligible impact because of caching. Accordingly, the weighting scheme assigns weights of 0.60, 0.30, and 0.10 to network, computing, and storage resources, respectively. Weighted aggregation produces a comprehensive entropy value that reflects the actual fluctuations of critical resources while suppressing minor fluctuations, thereby minimizing the overall entropy. These measurements form a three-dimensional entropy distribution, as shown in Figure 14.

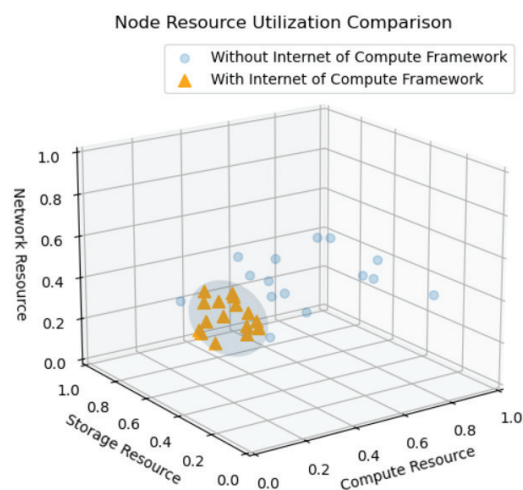


Figure 14. Distribution map of resource state entropy utilization rate.

The comparison of entropy values between the two environments verifies the improvement in resource stability achieved by the IoC.

Without the IoC architecture, the resource utilization distribution across nodes is highly dispersed. Some nodes remain under high load for extended periods, while others are relatively idle, resulting in highly imbalanced resource utilization. This dispersed distribution leads to a high overall level of node resource state entropy.

After introducing the IoC architecture, the resource utilization of each node converges to a relatively stable range, and the comprehensive state entropy is significantly reduced. This improvement results from the optimization of the task arrival rate λ and task service

time μ distributions through global resource visibility and intelligent scheduling.

For λ , the architecture distributes bursty tasks across multiple nodes through load balancing, thereby reducing per-node arrival fluctuations and bringing the number of task arrivals per unit time closer to the expected value.

For μ , the architecture matches tasks with the optimal resource combination, thereby reducing prolonged service times caused by resource contention or performance mismatches and concentrating task service times near the mean value, which decreases the variance σ .

Reduced fluctuations in λ make the probability distribution of resource utilization more concentrated, directly reducing network entropy and computing entropy. Lower variance in μ stabilizes resource occupation duration and further reduces computing entropy and storage entropy.

Experimental data show that after introducing the IoC architecture, the computing entropy, network entropy, and storage entropy of nodes decrease to varying degrees, significantly improving overall state stability. These results indicate that the IoC architecture enhances the predictability of resource states by optimizing task-level characteristics and provides a more reliable basis for fine-grained scheduling decisions.

Experiment 2: Comparative Performance Experiment between Traditional Internet Architecture and the IoC Architecture.

As shown in Figure 15, the experimental results reveal a clear performance gradient among the three groups in terms of TTFT and TPOT. The random distributed-node group under the IoC architecture performs best, followed by the random distributed-node group under the traditional Internet architecture, while the random centralized-node group under the traditional Internet architecture performs worst.

This ranking demonstrates that under the same deployment mode, the IoC architecture significantly outperforms the traditional Internet architecture. Under the same architecture, random distributed inference outperforms random centralized inference. Overall, the optimal performance achieved under the IoC architecture verifies the comprehensive advantages of deep computing-network convergence and entropy-balanced scheduling.

1) Formation Mechanism of Performance Differences.

- Group 1 (Random centralized nodes in the traditional Internet architecture): This group is limited by the computing-resource ceiling of a single node and lacks the capability to perceive node states through the Computing Identifier Gateway. As the number of concurrent requests increases, the queuing time of computing tasks grows exponentially, leading to rapid deterioration in TTFT. As the baseline control, this group reflects the performance boundary of random centralized inference in the traditional Internet architecture under the most unfavorable conditions.
- Group 2 (Random distributed nodes in the traditional Internet architecture): Although this group adopts random distributed parallel processing, it has limited inter-node network bandwidth and lacks resource state entropy perception and entropy-balanced scheduling. As described in Chapter 3, inference tasks are characterized by high λ (high-frequency arrivals) and low μ (short service time). In the traditional distributed architecture, gradient synchronization and intermediate result transmission become performance bottlenecks, causing Group 2 to exhibit significantly worse TPOT than Group 3 in long-context scenarios. This result demonstrates how the *computing-network decoupling* problem severely constrains distributed inference under the traditional Internet architecture.
- Group 3 (Random distributed nodes in the Internet of Computing architecture): This group constructs a global resource view through the Computing Identifier Gateway. The entropy-balanced scheduling algorithm dynamically allocates inference tasks to multiple low-entropy nodes for parallel processing and supports cross-domain high-speed data transmission. As described in Section 4.3, the control-plane separation design of the Interconnection Resource Layer enables the system to optimize both computing parallelism and communication efficiency. Its optimal performance across all test scenarios verifies the *synergistic gain* of random distributed parallelism and computing-network convergence.

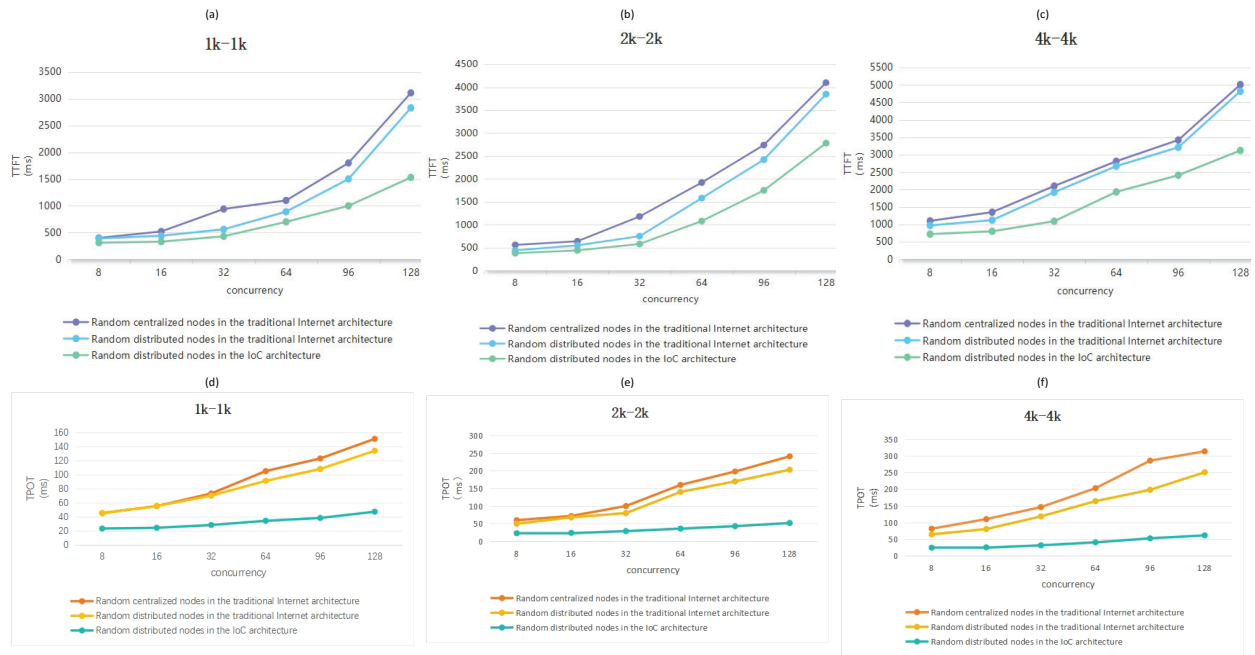


Figure 15. Performance of TTFT and TPOT under different scenarios.

2) Further Analysis in Long-Context and High-Concurrency Scenarios.

Across tests with different context lengths, longer contexts further amplify the architectural differences. When the input-output length increases to 4096–4096 tokens, the gap in TPOT between Group 2 and Group 3 widens significantly.

The demand for network bandwidth grows linearly with long-sequence transmission. The traditional Internet architecture lacks the ability to perceive and schedule network state entropy. Consequently, the cumulative effect of transmission delay caused by bandwidth constraints becomes more pronounced, increasing the proportion of communication time in Group 2 from 15% in short-context scenarios to approximately 35% in long-context scenarios. In contrast, Group 3 monitors network resource state entropy in real time through the Computing Identifier Gateway and preferentially assigns long-context tasks to low-entropy nodes, effectively limiting the increase in the proportion of communication time.

In high-concurrency scenarios, Group 3 exhibits optimal scalability. When concurrency rises from 8 to 128, the increase in its TTFT is only 52% of that in Group 2. This is because the IoC architecture distributes bursty requests across multiple low-entropy nodes through the entropy-balanced scheduling algorithm, avoiding

the exponential growth of queuing latency caused by single-node overload in traditional architectures.

VII. CONCLUSION

In summary, the IoC architecture theoretically solves the inherent defects of traditional architectures in heterogeneous resource scheduling, network state perception, and precise task matching through quantitative perception of resource state entropy, dynamic entropy-balanced scheduling, and deep computing-network collaboration. The overall leading performance of Group 3 in the experimental results is an empirical manifestation of the above theoretical advantages in a real heterogeneous computing environment, providing solid architectural support for large-scale, high-concurrency, and low-latency inference services.

REFERENCES

- [1] D. Kreutz, F. Ramos *et al.*, “Software-defined networking: A comprehensive survey,” *Proceedings of the IEEE*, 2015, vol. 103, no. 1, pp. 14–76.
- [2] S. Kianpisheh and T. Taleb, “A survey on in-network computing: Programmable data plane and technology-specific applications,” *IEEE Communications Surveys & Tutorials*, 2023, vol. 25, no. 1, pp. 701–761.
- [3] L. Zhang, A. Afanasyev *et al.*, “Named data networking,” *ACM SIGCOMM Computer Communication Review*, 2014, vol. 44, no. 3, pp. 66–73.

- [4] D. Naylor, M. Mukerjee *et al.*, “XIA: Architecting a more trustworthy and evolvable Internet,” *ACM SIGCOMM Computer Communication Review*, 2014, vol. 44, no. 3, pp. 50–57.
- [5] D. Clark, “The design philosophy of the DARPA Internet protocols,” *ACM SIGCOMM Computer Communication Review*, 1988, vol. 18, no. 4, pp. 106–114.
- [6] X. Tang, C. Cao *et al.*, “Computing power network: The architecture of convergence of computing and networking towards 6G requirement,” *China Communications*, 2021, vol. 18, no. 2, pp. 175–185.
- [7] Y. Mao, C. You *et al.*, “A survey on mobile edge computing: The communication perspective,” *IEEE Communications Surveys & Tutorials*, 2017, vol. 19, no. 4, pp. 2322–2358.
- [8] M. Chiang and T. Zhang, “Fog and iot: An overview of research opportunities,” *IEEE Internet of Things Journal*, 2016, vol. 3, no. 6, pp. 854–864.
- [9] P. Castro, V. Ishakian *et al.*, “The rise of serverless computing,” *Communications of the ACM*, 2019, vol. 62, no. 12, pp. 44–54.
- [10] W. Li, B. Zhang *et al.*, “Internet of computing architecture: Next-generation network architecture for supporting cross-region computing resources interconnection based on entropy balance,” *Journal on Communications*, 2025, vol. 46, no. 9, pp. 1–16.
- [11] M. Armbrust, A. Fox *et al.*, “A view of cloud computing,” *Communications of the ACM*, 2010, vol. 53, no. 4, pp. 50–58.
- [12] P. Bosshart, D. Daly *et al.*, “P4: Programming protocol-independent packet processors,” *ACM SIGCOMM Computer Communication Review*, 2014, vol. 44, no. 3, pp. 87–95.
- [13] C. Hong, S. Kandula *et al.*, “Achieving high utilization with software-driven WAN,” *ACM SIGCOMM Computer Communication Review*, 2013, vol. 43, no. 4, pp. 15–26.
- [14] I. Foster, “The anatomy of the grid: Enabling scalable virtual organizations,” in *Proceedings of the First IEEE/ACM International Symposium on Cluster Computing and the Grid*, 2001, pp. 6–7.
- [15] K. Krauter, R. Buyya *et al.*, “A taxonomy and survey of grid resource management systems for distributed computing,” *Software: Practice and Experience*, 2002, vol. 32, no. 2, pp. 135–164.
- [16] P. Barham, B. Dragovic *et al.*, “Xen and the art of virtualization,” *ACM SIGOPS Operating Systems Review*, 2003, vol. 37, no. 5, pp. 164–177.
- [17] R. Buyya, C. Yeo *et al.*, “Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility,” *Future Generation Computer Systems*, 2009, vol. 25, no. 6, pp. 599–616.
- [18] Q. Zhang, L. Cheng *et al.*, “Cloud computing: State-of-the-art and research challenges,” *Journal of Internet Services and Applications*, 2010, vol. 1, no. 1, pp. 7–18.
- [19] X. Wang, X. Ren *et al.*, “Net-in-AI: A computing-power networking framework with adaptability, flexibility, and profitability for ubiquitous AI,” *IEEE Network*, 2021, vol. 35, no. 1, pp. 280–288.
- [20] R. Mittal, A. Shpiner *et al.*, “Revisiting network support for RDMA,” in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, 2018, pp. 313–326.
- [21] Y. Zhu, H. Eran *et al.*, “Congestion control for large-scale RDMA deployments,” *ACM SIGCOMM Computer Communication Review*, 2015, vol. 45, no. 4, pp. 523–536.
- [22] Y. Li, R. Miao *et al.*, “HPCC: High precision congestion control,” in *Proceedings of the ACM Special Interest Group on Data Communication*, 2019, pp. 44–58.
- [23] X. Wang, X. Duan *et al.*, “Computing-aware network (CAN): A systematic design of computing and network convergence,” *Frontiers of Information Technology & Electronic Engineering*, 2024, vol. 25, no. 5, pp. 633–644.
- [24] C. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, 1948, vol. 27, no. 3, pp. 379–423.
- [25] D. Clark, J. Wroclawski *et al.*, “Tussle in cyberspace: Defining tomorrow’s Internet,” *IEEE/ACM Transactions on Networking*, 2005, vol. 13, no. 3, pp. 462–475.
- [26] A. Chiu, G. Choudhury *et al.*, “Architectures and protocols for capacity-efficient, highly dynamic, and highly resilient core networks [invited],” *Journal of Optical Communications and Networking*, 2012, vol. 4, no. 1, pp. 1–14.
- [27] O. Gerstel, M. Jinno *et al.*, “Elastic optical networking: A new dawn for the optical layer?” *IEEE Communications Magazine*, 2012, vol. 50, no. 2, pp. S12–S20.
- [28] A. Beloglazov, J. Abawajy *et al.*, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” *Future Generation Computer Systems*, 2012, vol. 28, no. 5, pp. 755–768.
- [29] S. Tang, Y. Yu *et al.*, “A survey on scheduling techniques in computing and network convergence,” *IEEE Communications Surveys & Tutorials*, 2024, vol. 26, no. 1, pp. 160–195.
- [30] M. Kim, J. Jang *et al.*, “Distributed task offloading and resource allocation for latency minimization in mobile edge computing networks,” *IEEE Transactions on Mobile Computing*, 2024, vol. 23, no. 12, pp. 15 149–15 166.

BIOGRAPHIES



more than 20 invention patents. Email: liwei3@caict.ac.cn

Li Wei is a professor of engineering. She currently works at the China Academy of Information and Communications Technology (CAICT), Beijing, China, where she has been engaged in research on cloud computing, artificial intelligence, making significant contributions to the establishment of internet of compute (IoC) architecture. She has published more than ten academic papers and holds



He has published more than ten SCI/EI academic papers. Email: zhaoweibo@caict.ac.cn

Zhao Weibo graduated from Beijing University of Posts and Telecommunications, Beijing, China, with a master’s degree. He is currently a cloud computing researcher and engineer at the China Academy of Information and Communications Technology (CAICT), Beijing, China. His main research interests include cloud computing, high-performance networking, and cloud-network integration.



copyrights, and 1 invention patent. Email: wangrunyan@caict.ac.cn

Wang Runyan received his M.S. degree from China Academy of Telecommunication Technology in 2024. He is currently an engineer at the China Academy of Information and Communications Technology (CAICT), Beijing, China. His research fields include computing power network, computing power identification, and cloud operating systems. He has published 5 papers in core journals, 4 software



Sang Liu graduated from Xidian University, Xi'an, China, with a master's degree and currently works at the China Academy of Information and Communications Technology (CAICT), Beijing, China, as an engineer. Her main research areas are cloud computing, high-performance networks, and internet of compute, and she has published several EI-indexed papers. Email: sangliu@caict.ac.cn



“Task and Resource Collaborative Orchestration and Scheduling Algorithm Based on Computing Resource Interconnected Network” at an international conference. Email: huangjinfei@caict.ac.cn

Huang Jinfei graduated from City University of Hong Kong, Hong Kong, China, with a master's degree. He is currently an intermediate engineer at the China Academy of Information and Communications Technology (CAICT), Beijing, China. His main research interests include artificial intelligence and cloud computing. His major academic achievements include publishing the paper



Sun Songlin, Ph.D., is a professor at Beijing University of Posts and Telecommunications, Beijing, China. His main research areas are wireless communication, multimedia technology, and artificial intelligence. Email: slsun@bupt.edu.cn



Liu Rumeng is an engineer at the China Academy of Information and Communications Technology (CAICT), Beijing, China. He is responsible for advancing initiatives in internet of compute and cloud-native artificial intelligence. Email: liuruming@caict.ac.cn



networks. He has published numerous SCI and EI papers. Email: suyue1@caict.ac.cn

Su Yue is a senior engineer at the China Academy of Information and Communications Technology (CAICT), Beijing, China. He holds a master's degree in engineering from Beijing University of Posts and Telecommunications, Beijing, China. His current research focuses on artificial intelligence, cloud computing, computing power, cloud networks, internet of compute, and high-performance



mafei@caict.ac.cn

Ma Fei is a senior engineer and holds a Ph.D. from Beijing Jiaotong University, Beijing, China, and currently works at the China Academy of Information and Communications Technology (CAICT), Beijing, China. His main research areas are cloud computing, artificial intelligence, internet of compute, and high-performance networks. He has published numerous EI papers. Email: